

Foundations of Robotics and Simulation

Prepared By

Dr. Islam Zakaria

Lecturer at the Information Technology Department, FCAIH, Capital University

Learning Objectives

- Define robotics through the lens of Computer Science
- Identify the triad of robotic components: Sensing, Computation, Actuation
- Classify robots by application domain and morphological structure
- Analyze the cyber-physical nature of robotic systems
- Evaluate computational challenges in robotics: Latency, Uncertainty, Real-time Constraints
- Compare traditional Sense-Plan-Act vs. modern End-to-End architectures
- Understand why simulation is indispensable in modern robotics R&D
- Analyze how software-level bugs manifest as physical failures
- Identify safety mechanisms in autonomous systems

What Exactly is a Robot? (The CS Perspective)

□ Formal Definition:

A robot is a real-time, autonomous software system embedded within a mechanical body, characterized by its ability to establish and maintain a closed-loop feedback system between **environmental perception** and **physical actuation**.

□ Examples:

- Human-looking machines from movies
- Self-driving cars
- Roomba vacuum cleaners
- Factory robot arms

What Exactly is a Robot? (The CS Perspective)

❑ The Feedback Threshold:

- System **WITH** feedback = Robot (adaptive, responsive)
- System **WITHOUT** feedback = Machine/Appliance (deterministic, fixed behavior)

❑ Illustrative Example:

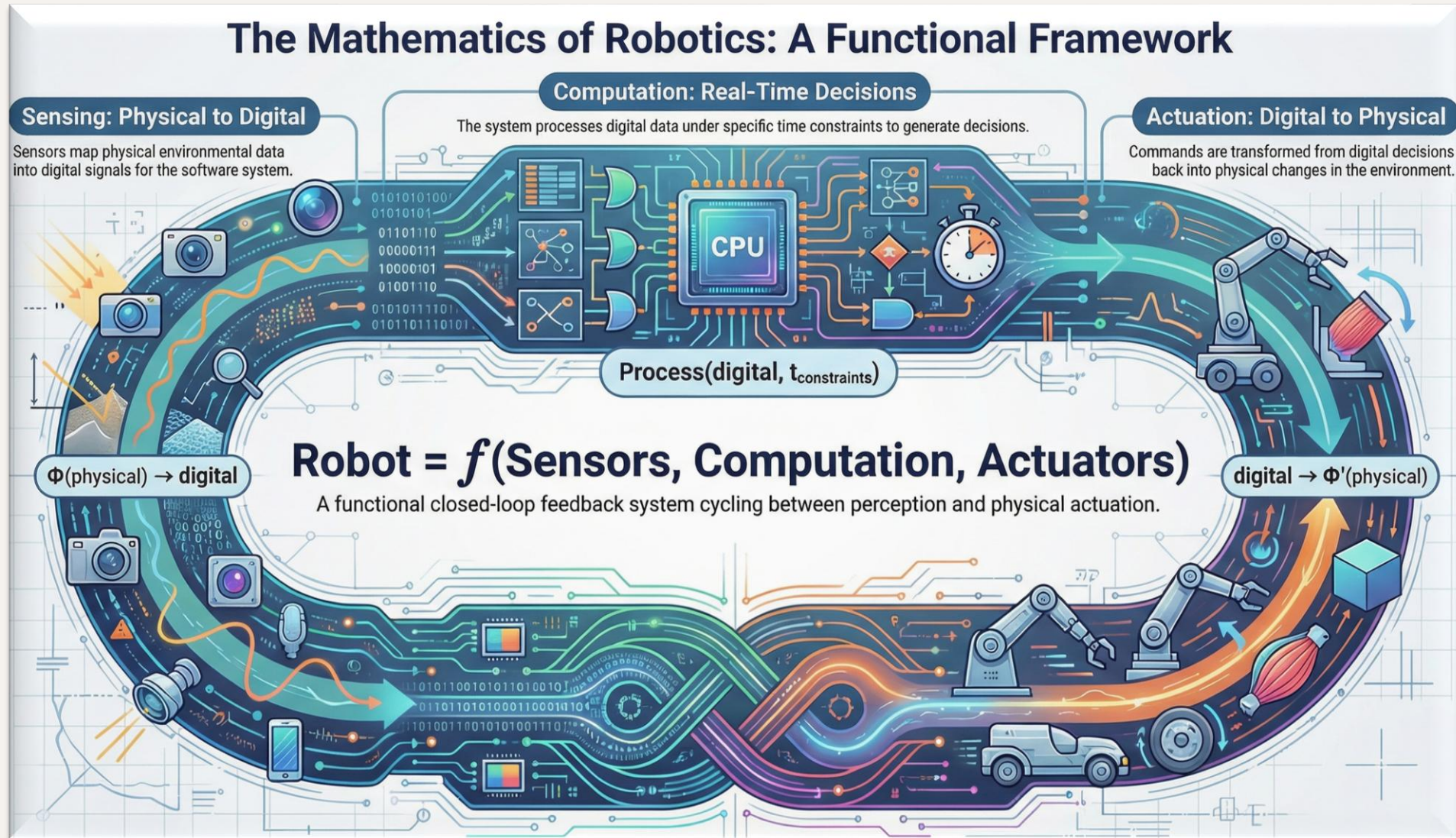
- **Robot:** Autonomous vacuum (**Roomba**) that navigates based on sensor input
- **Machine:** Programmable washing machine with fixed cycles

❑ Critical Difference: The vacuum adapts to furniture placement; the washer executes predetermined patterns

Think of a Robot Like a Human

Human Part	Robot Part	What It Does
Eyes/Ears	Sensors	See and hear the world
Brain	Computer	Decide what to do
Arms/Legs	Actuators	Move and interact
Nervous System	Wiring/Code	Connect everything

The Core Triad: Sensing → Computation → Actuation



Example

Automatic Doors

1. **Sensor:** Feels you coming (pressure mat or motion detector)
2. **Computer:** Decides to open the door
3. **Actuator:** Motor that opens the door

This Is a simple robot! It senses → thinks → acts

Sensing Layer (The "Eyes and Ears"):

Physical World → Analog Signals → Digital Data

- ❑ **What it does:** Turns real-world things into computer data
- ❑ **Examples:**
 - **Camera:** Takes pictures → turns into digital files
 - **Distance sensor:** Measures how far something is → gives a number
 - **Compass:** Finds direction → gives "North/South/East/West"
- ❑ **Simple Test:** **Close your eyes and try to walk across the room. Hard, right? That's what a robot without sensors is like!**

Computation Layer (The "Brain"):

Digital Data → Algorithms → Decisions

- ❑ **What it does:** Takes sensor data and decides what to do
- ❑ **Think of it like:** Your brain deciding "If I see a red light, I should stop"
- ❑ **Real-time Requirement:** **Correctness = Logical accuracy + Temporal precision**
- ❑ **Example Stack:**
 - **High-level:** Path planning (seconds resolution)
 - **Mid-level:** Trajectory generation (milliseconds)
 - **Low-level:** Motor control (microseconds)

Actuation Layer (The "Muscles"):

Digital Commands → Physical Force/Movement

- ❑ **What it does:** Turns computer commands into real movement
- ❑ **Examples:**
 - **Motor:** Spins wheels or arms
 - **Speaker:** Makes sounds
 - **Light:** Turns on/off
- ❑ **Challenge:** Overcoming physical inertia and environmental resistance

What Makes Robots DIFFERENT from Regular Computers?

Feature	Your Laptop	A Robot
Environment	Sits on desk	Moves in unpredictable world
Input	Keyboard/mouse clicks	Camera images, sensor readings
Output	Screen display	Physical movement
Consequences of Crash	Program closes	Robot crashes into wall!
Data	Exact (5 = exactly 5)	Noisy (5cm = somewhere near 5cm)
Key Difference	Computers live in the "digital world" (perfect, predictable)	Robots live in the "physical world" (messy, unpredictable)

The Three "Ages" of Robots

❑ Age 1: The Dumb Robots (1960s-1980s)

Input: None → Processing: Playback → Output: Repetitive motion

- **Technology:** Relay logic, fixed sequences
- **Limitation:** Zero adaptability, blind operation

❑ Age 2: The "Good Guessers" (1990s-2010s)

Input: Noisy sensors → Processing: Statistical filters → Output: Informed actions

- **Breakthrough:** Bayesian filtering, SLAM (Simultaneous Localization and Mapping)
- **Advancement:** Handling uncertainty, building environmental models

❑ Age 3: The Learners (2020s-Now)

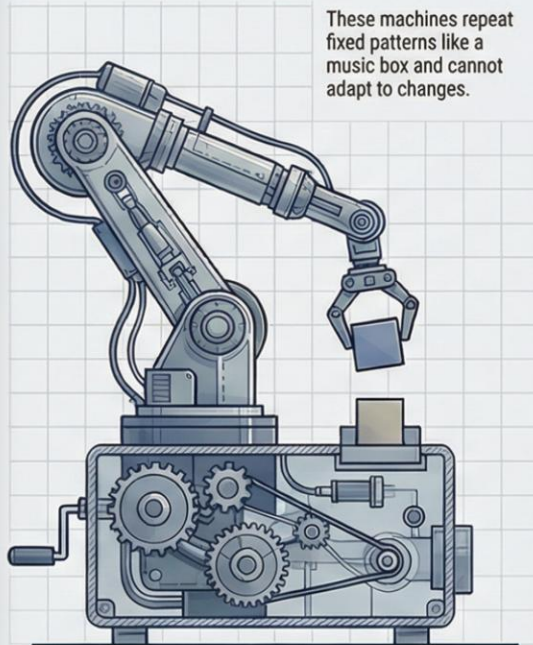
Input: Rich sensor data → Processing: Neural networks → Output: Learned behaviors

- **Technology:** Deep Reinforcement Learning, Simulation training
- **Revolution:** End-to-end learning, transfer from simulation to reality

The Three "Ages" of Robots (Cont.)

The Three Ages of Robot Intelligence

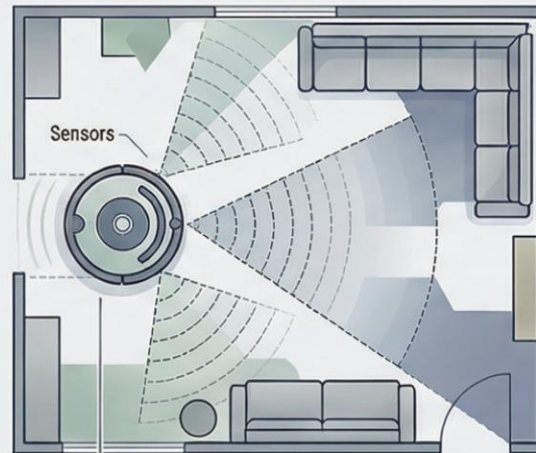
Age 1: The "Dumb Robots" (1960s–1980s)



These machines repeat fixed patterns like a music box and cannot adapt to changes.

Age 2: The "Good Guessers" (1990s–2010s)

Robots began using statistics to handle uncertainty and map their environments.

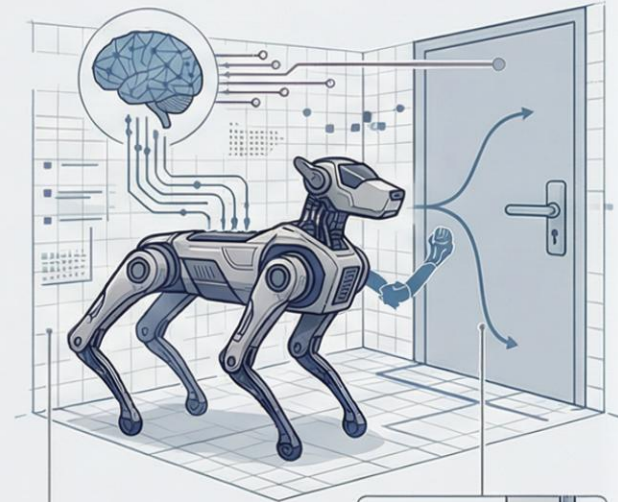


Roomba Vacuum Mapping

A practical application of robots making "good guesses" about their surroundings.

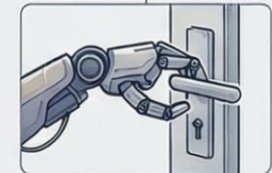
Age 3: The Learners (2020s–Present)

Modern robots learn through trial and error, similar to a human baby.



SIMULATION-BASED TRAINING

Robots practice movements in computer simulations before attempting them in reality.



Door-Opening Robot Dogs

Advanced machines that master complex physical interactions through practice.

Robotics vs. Traditional Software Engineering

□ Environmental Context:

- **Traditional Software:** Operates in controlled, deterministic digital environments

Web Server: Request → Process → Response

All interactions are through defined APIs

- **Robotics Software:** Operates in open, unpredictable physical environments

Mobile Robot: Sensor reading → Process → Actuation

Environment provides unconstrained, noisy inputs

□ Data Integrity:

- **Traditional:** Digital purity

distance = 5.0 # Always exactly 5.0

- **Robotics:** Probabilistic reality

distance = Normal($\mu=5.0$, $\sigma=0.2$) # 5.0 ± uncertainty

Robotics vs. Traditional Software Engineering (Cont.)

❑ Failure Consequences:

- **Traditional:** "Blue screen" → restart service
 - Cost: Downtime, data loss
- **Robotics:** "Physical crash" → hardware damage, potential injury
 - Cost: Repair expenses, safety incidents, liability

❑ Illustrative Case:

- **Web Application Bug:** Memory leak causes server restart
 - Impact: 5 minutes of downtime, user inconvenience
- **Surgical Robot Bug:** Control loop instability causes overshoot
 - Impact: Tissue damage, patient harm

Robotics vs. Artificial Intelligence

- ❑ **Pure AI:** Information → Processing → Output (information)
- ❑ **Robotics:** Physical input → Processing → Physical output
- ❑ **Temporal Constraints:**
 - **AI Systems:** Can utilize computational luxury
 - **Chess engines:** No real-time pressure, stronger with more depth
 - **LLMs:** Seconds for complex responses
 - **Robotic Systems:** Bound by physical dynamics
 - **Drone stabilization systems:** must adjust rotor speeds within **2–5 ms**.
 - **Autonomous car:** Must decide brake application within **100ms** at highway speeds
- ❑ **Spatial Reasoning:**
 - **AI Focus:** Semantic understanding, pattern recognition
 - **Robotics Requirement:** Geometric reasoning, physics prediction

Robotics vs. Embedded Systems

□ Complexity Scale:

➤ **Embedded Systems: Single-variable control**

- **Thermostat: Measure $T \rightarrow$ Compare to setpoint $\rightarrow$ Activate heater**
- **Single input, single output, linear relationship**

➤ **Robotic Systems: Multi-variable, coupled control**

- **Humanoid robot: 20+ joints $\times$ (position, velocity, torque)**
- **Non-linear, coupled dynamics**

Levels of Autonomy: Spectrum of Human Involvement

❑ Level 0: Manual Operation

- **Description:** Full human control with no automated assistance.
- **Example:** Aerial photography drone controlled entirely by a handheld transmitter (no auto-hover or GPS assist)
- **Software Role:** the system simply forwards user inputs to motors/actuators with no decision making.

❑ Level 1-2: Assisted Autonomy

- **Description:** System handles specific sub-tasks.
- **Example 1:** Drone auto-leveling (maintains orientation)
- **Example 2:** Car lane-keeping assist
- **Software Challenge:** Human-machine handoff protocols

Levels of Autonomy: Spectrum of Human Involvement (Cont.)

❑ Level 3-4: Conditional Autonomy

- **Description:** System handles full operation in defined conditions
- **Example:** Mercedes-Benz Drive Pilot on certified highway segments.
- **Limitation:** Still depends on a human fallback when the system encounters conditions outside its operational design domain (unexpected road events).
- **Critical Issue:** the system may request a takeover when the human driver is not fully attentive or ready, causing delayed reactions and potential safety risks.

❑ Level 5: Full Autonomy

- **Description:** Complete independence in all conditions
- **Example:** Hypothetical general-purpose home robot
- **Software Requirement:** Comprehensive edge-case handling
- **Current Status:** Not yet achieved for general applications

Real-Time Computing: The Heartbeat of Robotics

❑ Definition of "Real-Time":

Correctness = Logical Correctness + Temporal Correctness

➤ Not: "Fast" But: "Predictable timing"

❑ Deadline Types:

1. **Hard Deadlines:** Miss → Catastrophic failure

Anti-lock Braking System (ABS):

- **Required:** Compute slip ratio, adjust brake pressure every 5ms
- **Consequence of miss:** Loss of control, accident

1. **Soft Deadlines:** Miss → Performance degradation

Mapping update

- **Desired:** Update occupancy grid every 100ms
- **Consequence of occasional miss:** Slightly outdated map

Latency vs. Jitter

- ❑ **Latency:** Total delay to response
- ❑ **Jitter:** Variation in latency (deviation from periodicity)
- ❑ **Example:**

Camera processing pipeline

- **Ideal:** Process every frame in exactly 33ms (30Hz)
- **Reality:** **Frame 1: 30ms, Frame 2: 40ms, Frame 3: 35ms**
- **Latency:** **~35ms** average
- **Jitter:** **±5 ms variation** → **causes control instability**

Timing is EVERYTHING in Robotics

❑ Two Types of Timing Problems:

- **Being Slow:** Reacting too late
- **Being Inconsistent:** Sometimes fast, sometimes slow (even worse!)

❑ Sources of Timing Problems:

1. **Garbage Collection (Java/Python):** Non-deterministic pauses
2. **Context Switching:** OS interrupts computational flow
3. **Memory Access:** Cache misses cause variable timing
4. **I/O Operations:** Disk/network access delays

Example with Illustrative Calculation

From MPH to m/s : $\times (1609/3600)$

Autonomous vehicle at 60 mph (26.8 m/s):

- 1. Perception latency (50ms):** This is the time it takes **sensors + neural networks** to see and understand what's happening:
(**Detect obstacle, Classify it, Localize it in 3D space**).
- 2. Planning latency (100ms):** This is the time required for the AI to predict future motion.
- 3. Actuation latency (20ms):** This is how long the vehicle's physical hardware takes to communicate to actuators.
- 4. Total latency:** 170ms

Example with Illustrative Calculation (Cont.)

5. Distance traveled: $26.8 \times 0.170 = 4.56$ meters.

- So the car moves 4.56 meters before any brake happens
- Result: if an obstacle appears 4.56 meters ahead
 - The car will only begin reacting at that moment.
 - Meaning it's virtually impossible to avoid the obstacle.
 - This shows why latency reduction is critical in autonomous vehicles
 - Even small delays (tens of milliseconds) matter a lot at highway speeds.

How Robots "Think" - The Control Loop

❑ Imagine You're Walking While Texting:

1. **Look up** (sensing: where am I?)
2. **Notice you're drifting left** (processing)
3. **Step right to correct** (acting)
4. **Repeat!**

❑ That's EXACTLY What Robots Do (**Loop Forever**):

1. **Read Sensors** (Where am I?)
2. **Compare to Goal** (Where do I want to be?)
3. **Calculate Error** (How far off am I?)
4. **Send Commands to Fix Error** (Move to correct!)
5. **Go back to Step 1**

This is Called "**CLOSED LOOP CONTROL**"

How Robots "Think" - The Control Loop (Cont.)

- ❑ Key Idea: **Constantly checking and adjusting**
- ❑ Like: **Cruise control in a car that maintains speed**
- ❑ Without This Loop (**Open Loop**):
 1. **Robot gets a command: "Move 10 feet"**
 2. **Moves without checking**
 3. **Hits an obstacle, only moves 9 feet**
 4. **Robot thinks: "I'm at 10 feet!" (Wrong!)**
- ❑ Illustrative Example: Temperature Control
 - **Open-loop**: **Heater on for 10 minutes regardless of actual temperature**
 - **Closed-loop**: **Thermostat measures temperature, adjusts heating accordingly**

Control Systems: Open-Loop vs. Closed-Loop

Aspect	Open-Loop Control	Closed-Loop Control
Feedback	No feedback loop.	Includes a feedback loop .
Control Action	Independent of the output.	Dependent on the output (Correction).
Components	Controller and Actuator.	Controller, Actuator, Sensor , and Comparator .
Accuracy	Less accurate; prone to errors .	Highly accurate; self-correcting.
Complexity	Simple and easy to design.	Complex and harder to design.
Stability	Generally stable.	Can become unstable if not tuned (Oscillation).
Cost	Cheaper.	More expensive (due to sensors).
Example	Toaster, Washing machine.	Robot arm with encoders , Air conditioner.

The Classic Robot "Thinking" Pattern

❑ Sense → Plan → Act (The Old Way)

Think of Making Coffee:

- **Sense:** Look at coffee machine (is there water? coffee?)
- **Plan:** Figure out steps (add water → add coffee → press start)
- **Act:** Do the steps

❑ Problems With This:

- **Slow!** Must finish sensing before planning, planning before acting
- **Like texting while walking:** Look down (sense) → Read text (plan) → Look up (act) → CRASH into pole!

❑ The New Way: Do Everything at Once!

- **Sense → plan → act simultaneously** (Like walking normally: Your brain does all three continuously)

Robot "Brains" Need Special Software

❑ Why Not Just Use Windows/Mac?

- Regular computers do many things at once (browser, music, updates)
- Robots need predictable timing
- **Imagine if your car's brake computer decided to do Windows Update while driving!**

❑ Enter: ROS (Robot Operating System)

- **NOT** really an operating system (runs on Linux/Windows)
- **It is a "traffic director" for robot software**
- Makes sure messages get where they need to go, **ON TIME**

Robot "Brains" Need Special Software

- ❑ **ROS Concept:** Different "Mini-Programs" Working Together
- ❑ **Eyes Program:** Just handles cameras
- ❑ **Navigation Program:** Just plans paths
- ❑ **Motor Program:** Just controls motors
- ❑ **ROS connects them all!**
- ❑ **Benefit:** If the "Eyes Program" crashes, the robot can still stop safely
(Motor Program keeps working)

Sensors Are Not Perfect!

❑ Real Sensors vs. Perfect Sensors:

❑ Perfect World:

- **Distance sensor:** "Object is exactly 5.0 meters away"

❑ Real World:

- **Distance sensor:** "Object is... maybe 4.9m? Or 5.1m? Let me check again... now 5.2m?"

❑ Types of Sensor Problems:

1. The Jitters (Noise)

- Like trying to read a scale while standing on it
- Number keeps bouncing around
- **Solution: Take average of many readings**

Sensors Are Not Perfect!

2. Always Wrong (Bias)

- Like a watch that's always 5 minutes fast
- Consistently wrong in same way
- **Solution: Calibrate (tell robot "this is actually 0")**

3. Sometimes Crazy Wrong (Outliers)

- Like your GPS suddenly saying you're in the ocean
- Rare but happens
- **Solution: Ignore crazy readings**

4. Smart Trick: Use MULTIPLE sensors!

- GPS says: "You're here"
- Camera says: "You see this building"
- **Combined: Much more accurate!**

Where Should the Robot "Think"?

❑ Three Options for Robot Brains:

❑ Option 1: On the Robot (Edge Computing)

- **Like:** Thinking in your head
- **Pros:** **Fast! No delay**
- **Cons:** **Limited "brain power"**
- **Used for:** Emergency stops, balance

❑ Option 2: On Nearby Computer (Fog Computing)

- **Like:** Asking a friend nearby
- **Pros:** **More powerful than robot's brain**
- **Cons:** **Slight delay**
- **Used for:** Complex planning

Where Should the Robot "Think"? (Cont.)

❑ Option 3: In the Cloud

- **Like:** Calling an expert on the phone
- **Pros:** **SUPER powerful (thousands of computers!)**
- **Cons:** **Can lose connection**
- **Used for:** Learning from other robots worldwide

❑ Modern Approach: Use all three!

- **Emergency decisions:** On robot (must be fast!)
- **Route planning:** In cloud (needs lots of data)
- **Everything else:** In between



**Any
Questions**